Projet CSI

Sujet 4 – Plateforme de validation collaborative de pull requests (gamifiée)

GL2/3
2025/2026

Raed Jani
Adem Ayari
Mohamed Amine Barkati
AbdelAziz Mseddi

I- Introduction

Contexte

Dans le cadre du développement logiciel collaboratif, la revue de code constitue une étape cruciale du cycle de vie d'un projet. Elle permet de garantir la qualité du code, de détecter les erreurs en amont et de favoriser le partage de connaissances au sein d'une équipe.

Cependant, dans les équipes de développement, la répartition des revues de code est souvent inégale : certains développeurs se retrouvent surchargés tandis que d'autres sont sous-sollicités, ce qui nuit à la productivité globale et à la qualité des livrables.

C'est dans ce contexte que s'inscrit la Plateforme de validation collaborative de pull requests (gamifiée), réalisée dans le cadre du projet CSI (GL2/3) à l'INSAT.

Présentation du projet

Le projet consiste à concevoir et développer un outil interne destiné aux équipes de développement, dont l'objectif principal est d'assurer une répartition équitable et intelligente des revues de code.

La plateforme s'articule autour du concept de Pull Request (PR), une demande faite par un développeur pour que ses modifications de code soient examinées et intégrées dans la branche principale du projet.

L'outil permet aux développeurs de revendiquer une PR à faire réviser, puis suggère automatiquement les reviewers les plus appropriés en tenant compte de leur charge de travail actuelle et de leur historique de revues.

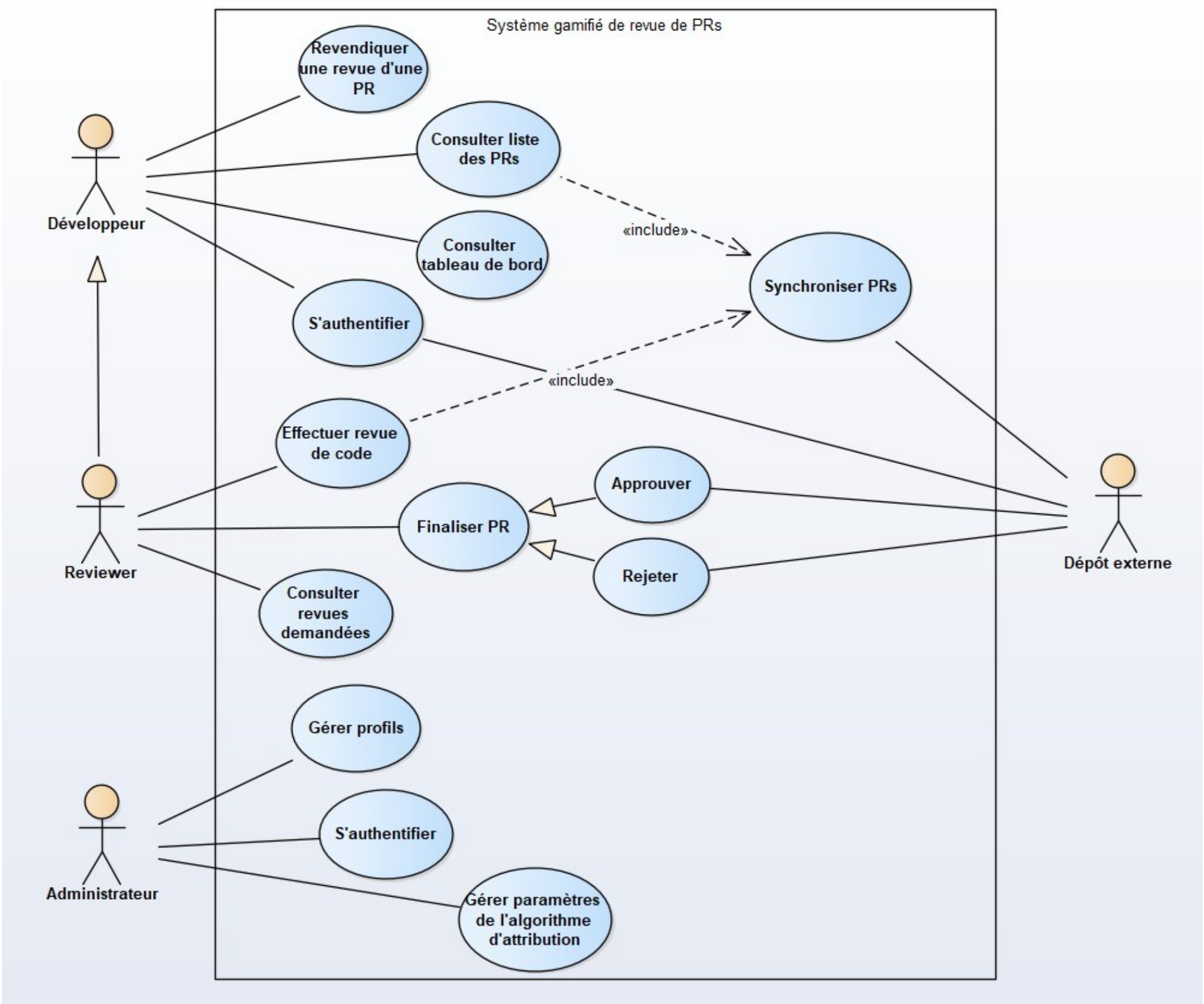
Afin d'encourager la participation active, un système de gamification est intégré : chaque revue effectuée rapporte des points, et des badges sont attribués selon les performances, rendant l'expérience à la fois motivante et engageante.

Objectifs

Les principaux objectifs de cette plateforme sont les suivants :

- Centraliser et synchroniser les pull requests depuis des dépôts externes (GitHub, GitLab, Bitbucket).
- Automatiser la suggestion de reviewers grâce à un algorithme d'attribution basé sur la charge de travail, l'historique des revues et le score des développeurs.
- Garantir une répartition équitable des revues au sein de l'équipe, évitant ainsi la surcharge de certains membres.
- Motiver les développeurs à s'impliquer dans le processus de revue via un système de points et de badges (gamification).

II- Analyse des besoins


Système gamifié de revue de PRs
Revendiquer une revue d'une PR
Consulter liste des PRs
Consulter tableau de bord
S'authentifier
Synchroniser PRs
«include»
«include»
Effectuer revue de code
Approuver
Finaliser PR
Rejeter
Consulter revues demandées
Gérer profils
S'authentifier
Gérer paramètres de l'algorithme d'attribution
Développeur
Reviewer
Administrateur
Dépôt externe

1) Description du cas <S'authentifier>:

Ce cas permet de créer un compte lié à un ou plusieurc comptes externes.

Pré-conditions : aucune.

Déroulement nominal :

- le Développeur crée son profil (nom, prénom, pseudo, ...).
- il ajoute son email pour recevoir des notifications.
- il ajoute un ou plusieurs connexion(s) à des comptes externes (GitHub, GitLab, GitBucket).
- chaque connexion nécessite une authentification externe.

Déroulement alternatif : Si l'utilisateur a déjà un compte on se contente du email et du mot de passe.

Post-conditions : Connexion établie avec le(s) compte(s). Chargement des PRs disponibles.

2) Description du cas <Consulter tableau de bord>

Ce cas permet au Développeur de consulter son tableau de bord.

Pré-conditions : être authentifié.

Déroulement nominal :

- le tableau de bord constitue la fenêtre principale du système après login, il y est dirigé automatiquement.
- le Développeur y trouve son profil minimal(photo, pseudo), la liste de Pull Requests et la liste de Revues en cours et en attente.
- pour les Reviewers la liste des revues contient aussi les revues demandées.

3) Description du cas <Consulter liste des PRs>

Ce cas permet de consulter la liste des Pull Requests créées par le Développeur.

Pré-conditions : être authentifié.

Déroulement nominal :

- le Développeur, après login, est redirigé automatiquement au tableau de bord.
- Le Développeur peut importer les dépôts qu'il souhaite depuis son compte externe.
- le tableau de bord contient ainsi la liste des PRs relatifs aux dépôts choisis et leurs dates d'ouverture (et éventuellement fermeture avec le reviewer associé).
- le développeur peut à chaque fois cliquer sur un bouton pour synchroniser ses PRs (synchronisation automatique à l'ouverture de la fenêtre).

4) Description du cas <Revendiquer une revue de PR>

Ce cas permet au Développeur de revendiquer une revue de PR.

Pré-conditions : en train de consulter les PRs.

Déroulement nominal :

- le Développeur clique sur une PR pour demander une revue.
- le système le dirige vers une fenêtre contenant les détails de la PR et lui suggère 5 Reviewers triés par l'algorithme d'attribution.
- le développeur choisit un ou plusieurs reviewers.

Post-conditions : le(s) reviewer(s) notifié(s) par e-mail.

5) Description du cas <Consulter revues demandées>

Ce cas permet au Reviewer de lister ses Revues en attente.

Pré-conditions : le Reviewer est authentifié.

Déroulement nominal :

- le Reviewer consulte dans le tableau de bord.
- dans la liste des Revues il trouve les Revues demandées (il peut filtrer pour une meilleure visibilité).

6) Description du cas <Effectuer une revue de la Pull Request>

Ce cas permet au Reviewer d'effectuer une Revue du code d'une PR.

Pré-conditions : le reviewer est authentifié et un Développeur lui demande une revue.

Déroulement nominal :

- le Reviewer choisit parmi la liste de Revues demandées.
- le Reviewer est redirigé vers une fenêtre contenant les informations de la Revue.
- il dispose d'un formulaire ou il laisse un commentaire detaillé sur la pull request.

Post-conditions :

- attribution de points au reviewer.

7) Description du cas <Finaliser une PR>

Ce cas permet au Reviewer de fermer une Pull Request.

Pré-conditions : le Reviewer est authentifié et a fait au moins une revue sur la Pull Request.

Déroulement nominal :

- le Reviewer choisit une Pull Request parmi celles dont une revue lui est attribuée.
- dans la fenêtre de la Pull Request il trouve deux boutons supplémentaires (soit approuver soit rejeter).
- cliquer l'un des deux finalise la Pull Request.

Post-conditions :

- le dépôt externe est notifié par la décision pour être à jour.
- le propriétaire de la Pull Request est notifié par email.
- attribution de points au Reviewer dans les deux cas.
- attribution de points au Developper au cas d'acceptation.

8) Description du cas <Gérer paramètre de l'algorithme d'attribution>

Ce cas permet à l'Administrateur de modifier les paramètres de l'algorithme d'attribution.

Pré-condition : Administrateur authentifié.

Déroulement nominal :

- le Reviewer modifie la pondération des paramètres de l'algorithme dans le score (charge, points, badges et note).
- il applique les nouveaux paramètres et confirme.

III- Design et conception structurelle

1. Les classes Entité (Entity / Modèle)

Rôle: Représenter les **concepts métier** et leurs données. Ce sont les "noms communs" du domaine.

Contenu: Des **attributs** (les données)

Ce qu'elles ne font PAS: Elles ne parlent ni à l'utilisateur ni à la base de données ni aux systèmes externes.

2. Les classes Interface (Boundary / GUI / Vue)

Rôle: Gérer tout ce qui est **affiché** à l'utilisateur et toutes les **interactions** avec lui (clics, formulaires, navigation).

Ce qu'elles ne font PAS: Elles ne prennent **aucune décision métier mais transmettent l'information au contrôlleur**.

Contenu:

- Des composants graphiques (boutons, listes, formulaires).
- Des méthodes qui réagissent aux événements utilisateur.

3. Les classes Contrôle (Controller)

Rôle: **Orchestrer** les cas d'utilisation. Elles font le pont entre ce que l'utilisateur veut faire (boundary) et ce que ça implique sur les données (entity + repository).

Contenu:

- De la **logique applicative** : enchaînement d'étapes, coordination entre plusieurs entités
- Des **règles métier transversales** qui ne rentrent dans aucune entité spécifique
- Des **appels aux repositories** pour sauvegarder/charger

Ce qu'elles ne font PAS: Elles n'affichent rien et ne stockent pas directement les données (elles délèguent aux repositories).

4. Les classes Repository (DAO / Persistance)

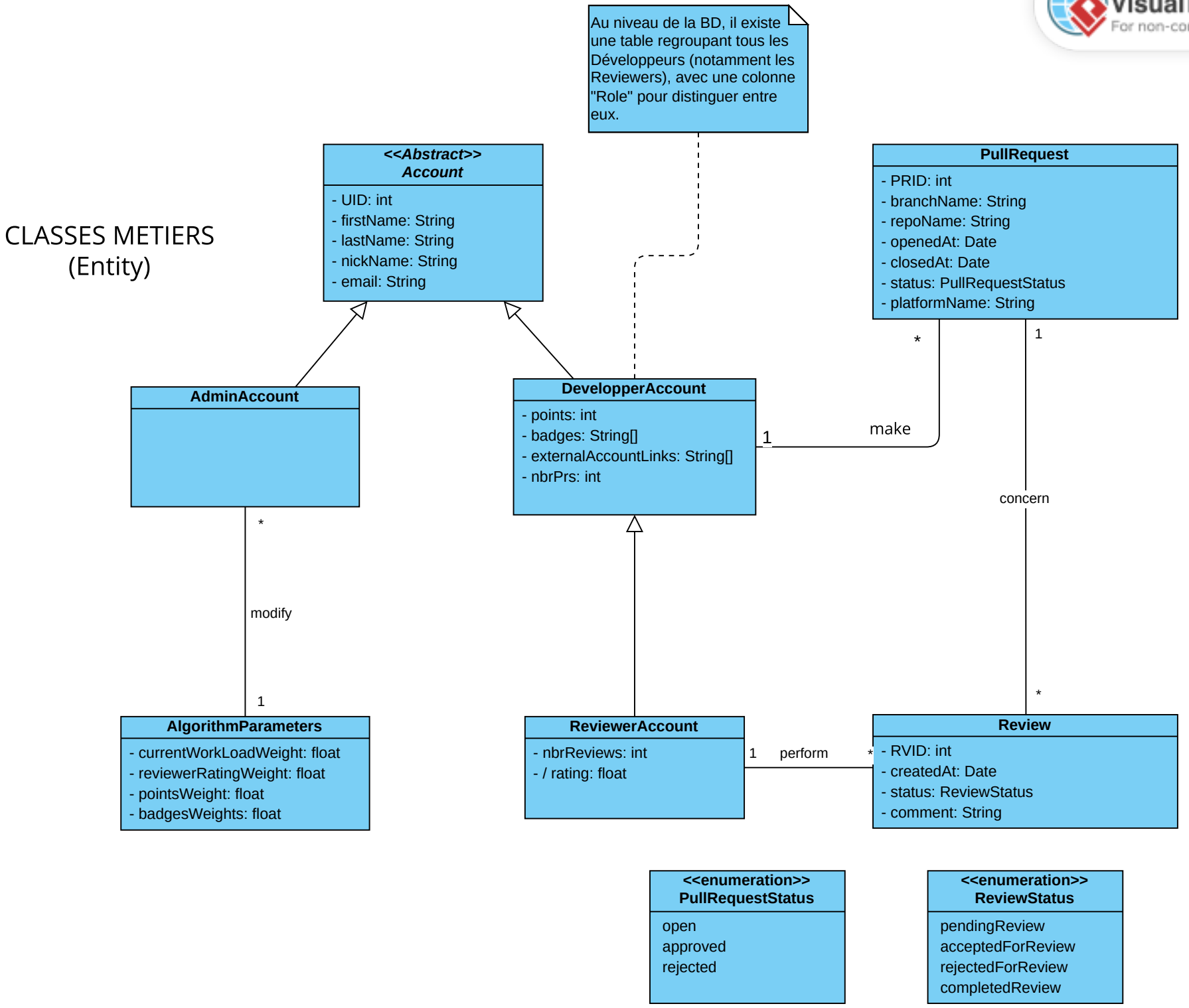
Rôle: Gérer la **persistance** : sauvegarder et récupérer les entités dans une base de données (ou un fichier, ou une API).

Contenu: Les requêtes SQL

Ce qu'elles ne font PAS: Aucune logique métier et il ne fait que stocker et retrouver.

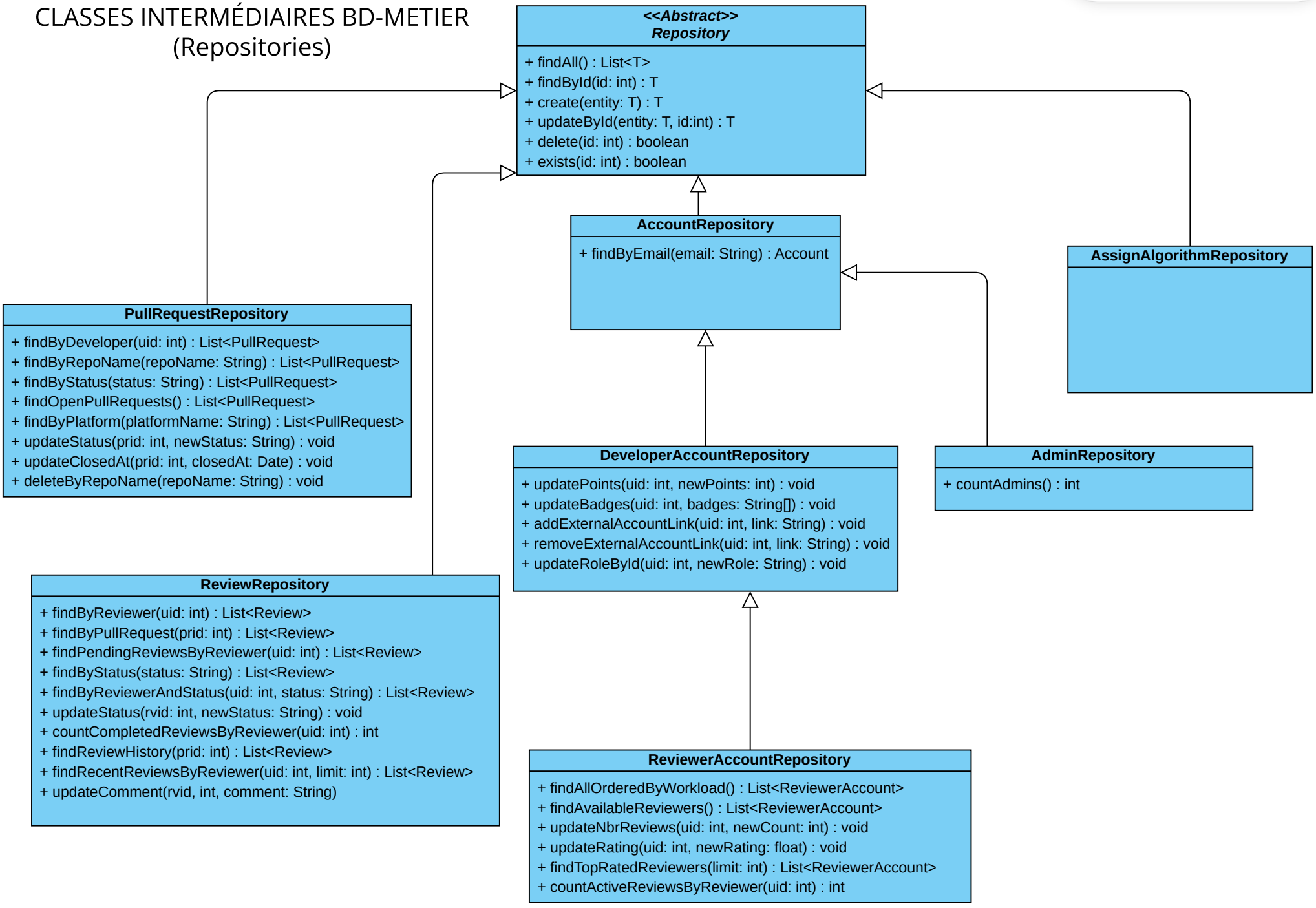
Pourquoi les séparer des entités ?

Parce que si on change de base de données (MySQL → MongoDB), on ne veut réécrire que les repositories, pas toutes les entités.


CLASSES METIERS
(Entity)
Au niveau de la BD, il existe une table regroupant tous les Développeurs (notamment les Reviewers), avec une colonne "Role" pour distinguer entre eux.
<<Abstract>>
Account
- UID: int
- firstName: String
- lastName: String
- nickName: String
- email: String
PullRequest
- PRID: int
- branchName: String
- repoName: String
- openedAt: Date
- closedAt: Date
- status: PullRequestStatus
- platformName: String
AdminAccount
DevelopperAccount
- points: int
- badges: String[]
- externalAccountLinks: String[]
- nbrPrs: int
make
concern
modify
AlgorithmParameters
- currentWorkLoadWeight: float
- reviewerRatingWeight: float
- pointsWeight: float
- badgesWeights: float
ReviewerAccount
- nbrReviews: int
- / rating: float
perform
Review
- RVID: int
- createdAt: Date
- status: ReviewStatus
- comment: String
<<enumeration>>
PullRequestStatus
open
approved
rejected
<<enumeration>>
ReviewStatus
pendingReview
acceptedForReview
rejectedForReview
completedReview

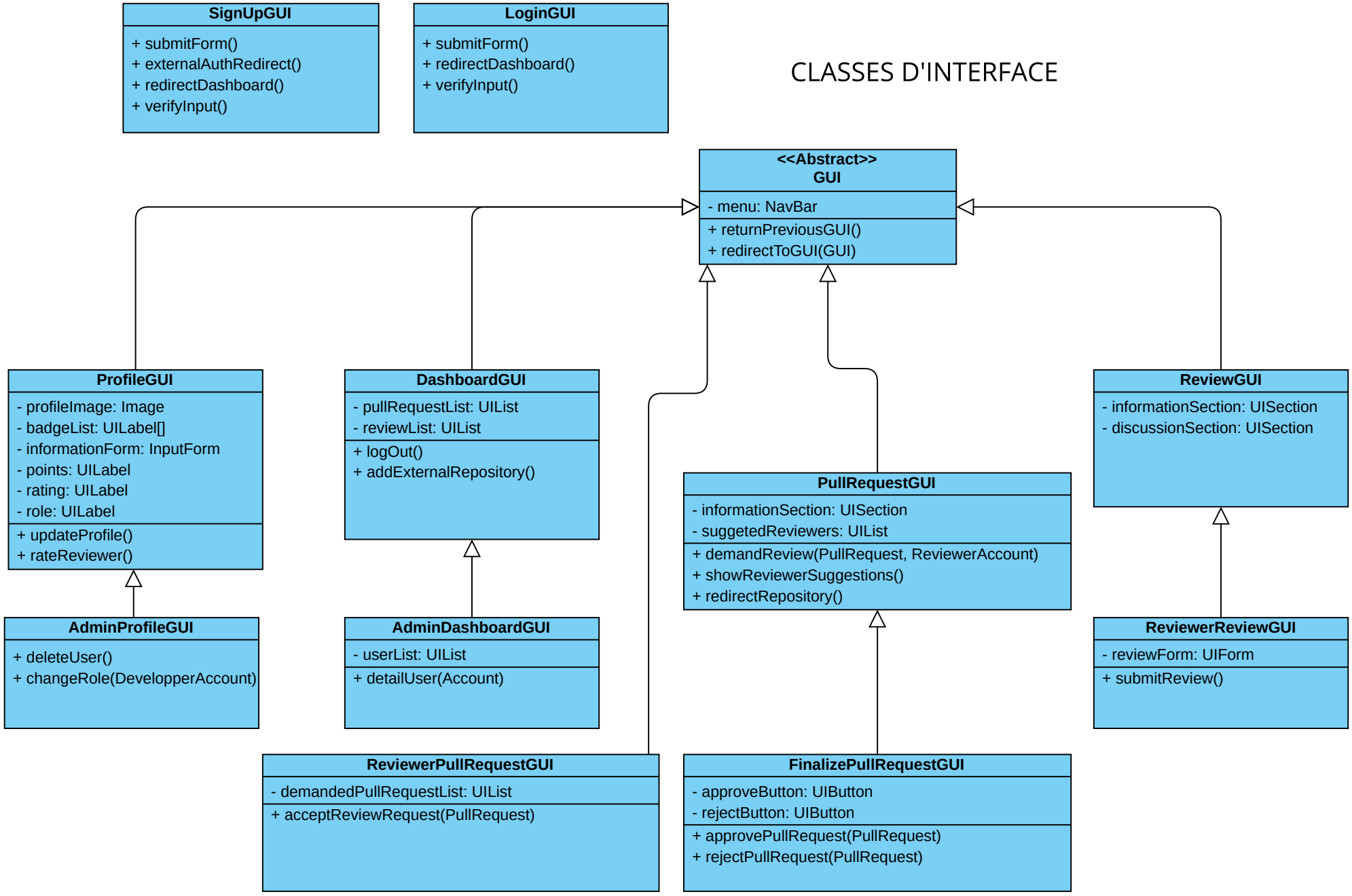
Made with Visual Paradigm
For non-commercial use

CLASSES INTERMÉDIAIRES BD-METIER (Repositories)



Made with Visual Paradigm
For non-commercial use

CLASSES D'INTERFACE



CLASSES DE CONTRÔLE
(Controller)

Made with Visual Paradigm
For non-commercial use

ReviewController

+ loadRequestedReviews(reviewer: ReviewerAccount) : List<Review>
+ getReviewDetails(rvid: int) : Review
+ submitReview(reviewId: int, comment: String) : void
+ acceptReviewRequest(pr: PullRequest) : void

PullRequestController

+ loadPullRequests(dev: DevelopperAccount) : List<PullRequest>
+ importRepository(dev: DevelopperAccount, repoName: String) : void
+ claimPullRequest(pr: PullRequest) : void
+ assignReviewers(pr: PullRequest, reviewers: List<ReviewerAccount>) : void
+ getPullRequestDetails(prid: int) : PullRequest
+ finalizePullRequest(prid: int, decision: String) : void

AttributionController

+ suggestReviewers(pr: PullRequest) : List<ReviewerAccount>
+ calculateScore(reviewer: ReviewerAccount, algo: AlgorithmParameters) : float
+ getCurrentAlgorithm() : AlgorithmParameters

AuthentificationController

+ login(email, password) : Account
+ signUp(accountData) : DevelopperAccount
+ linkExternalAccount(platformName, credentials) : boolean
+ logout() : void
+ verifyCredentials(email, password) : boolean

SynchronisationController

+ synchronizePRs(developer: DevelopperAccount) : List<PullRequest>
+ fetchExternalRepos(link: String) : List<String>
+ notifyExternalPlatform(pr: PullRequest, decision: String) : void

GamificationController

+ awardPoints(developper: DevelopperAccount, action: String) : void
+ checkBadgeConditions(reviewer: ReviewerAccount) : List<String>
+ grantBadge(reviewer: ReviewerAccount, badge: String) : void
+ updateRating(reviewer: ReviewerAccount, rating: float) : void

NotificationController

+ notifyReviewRequest(reviewer: ReviewerAccount, pr: PullRequest) : void
+ notifyPROwner(developer: DevelopperAccount, decision: String) : void
+ sendEmail(account: Account, subject: String, body: String) : void

DashboardController

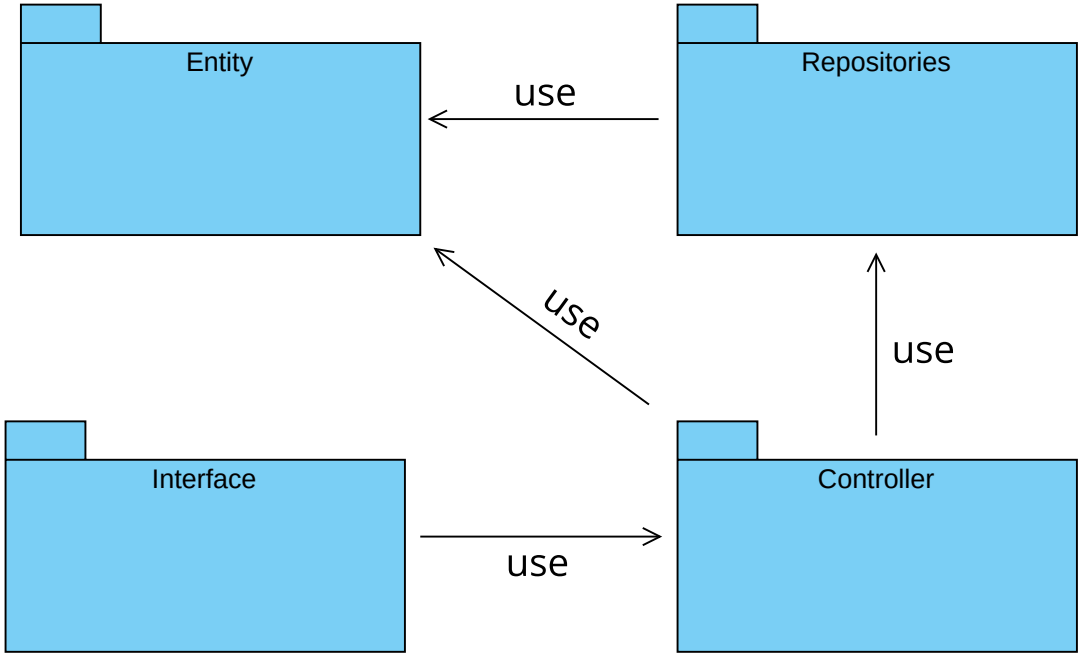
+ loadDashboard(account: Account) : DashboardData
+ loadProfile(account: Account) : ProfileData
+ loadUserList() : List<Account>
+ updateProfile(account: Account, data: ProfileData) : void

AdministrationController

+ updateAlgorithmParameters(params: AssignAlgorithm) : void
+ deleteUser(account: Account) : void
+ changeRole(dev: DevelopperAccount, newRole: String) : void
+ manageUsers() : List<Account>

Made with Visual Paradigm
For non-commercial use

Diagramme de packages



IV- Conception comportementale

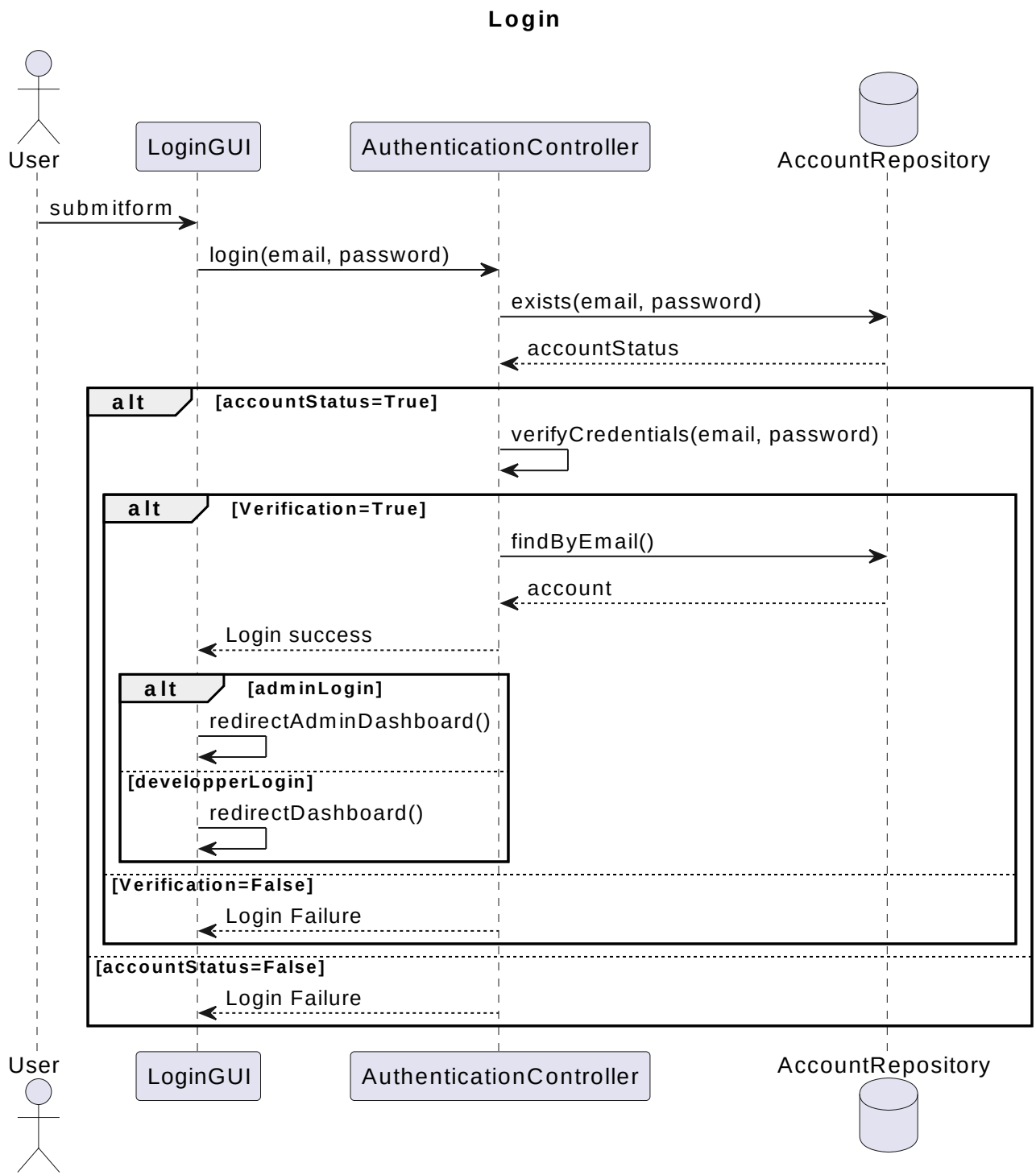
Après avoir défini la structure statique du système à travers les diagrammes de classes et de packages, cette section s'intéresse à son comportement dynamique : comment les objets interagissent-ils entre eux pour réaliser les cas d'utilisation ?
Pour modéliser ces interactions, nous utilisons les diagrammes de séquence UML, qui représentent l'enchaînement chronologique des messages échangés entre les objets participants (interfaces, contrôleurs, repositories et entités) lors de l'exécution d'un scénario.

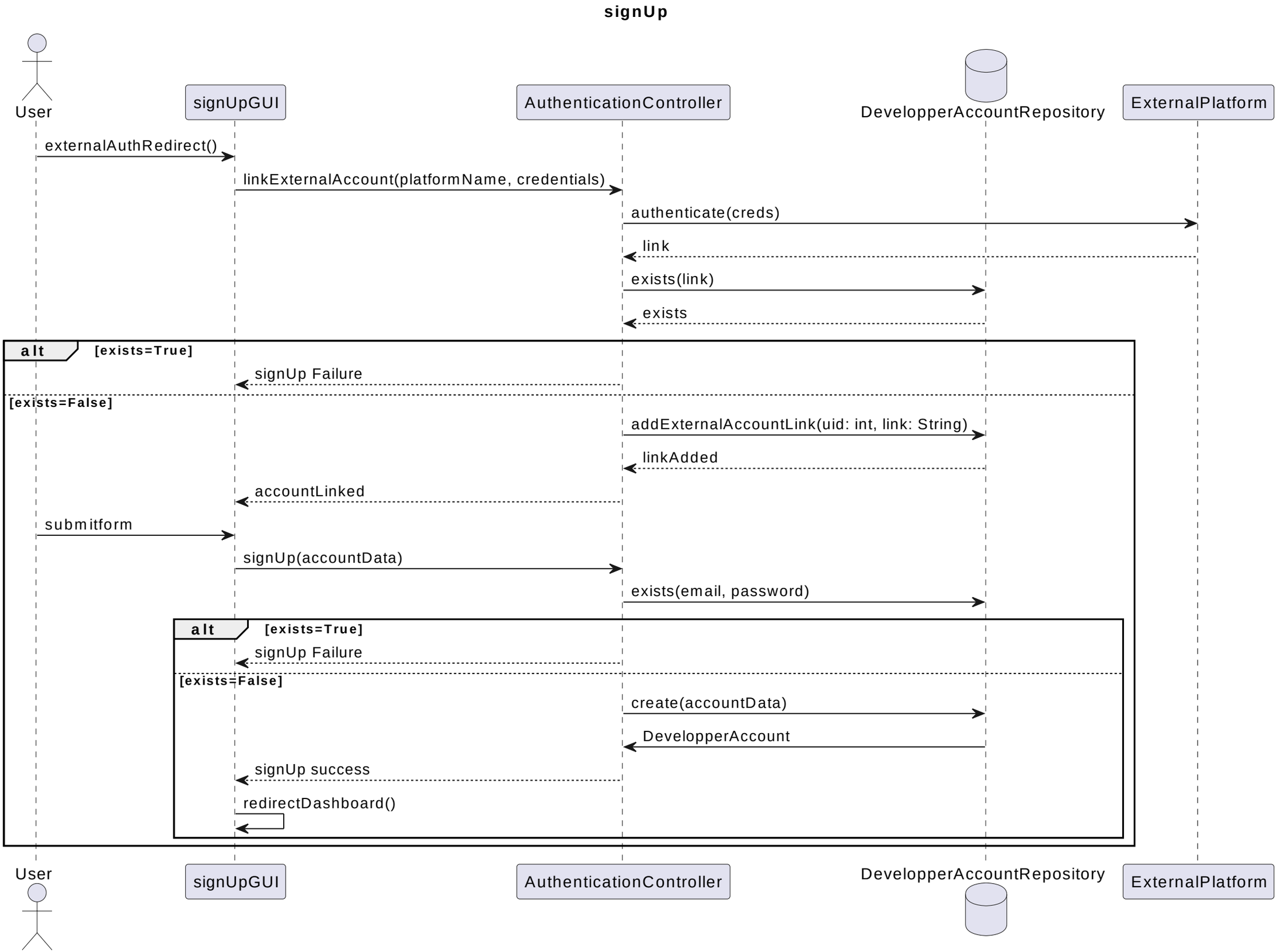
Parmi les cas d'utilisation identifiés dans l'analyse des besoins, nous avons sélectionné les scénarios suivants pour leur pertinence et leur représentativité :

- S'authentifier: Ce cas illustre le point d'entrée du système, avec l'intégration aux plateformes externes (GitHub, GitLab, Bitbucket) et la gestion du scénario alternatif d'échec de connexion.
- Revendiquer une revue d'une PR: Ce cas met en évidence le cœur fonctionnel du projet, l'algorithme d'attribution automatique des reviewers, qui constitue la valeur ajoutée principale de la plateforme.
- Effectuer une revue de code: Ce cas couvre l'interaction quotidienne du reviewer avec le système, incluant le chargement de l'historique de discussion et la soumission d'un commentaire.

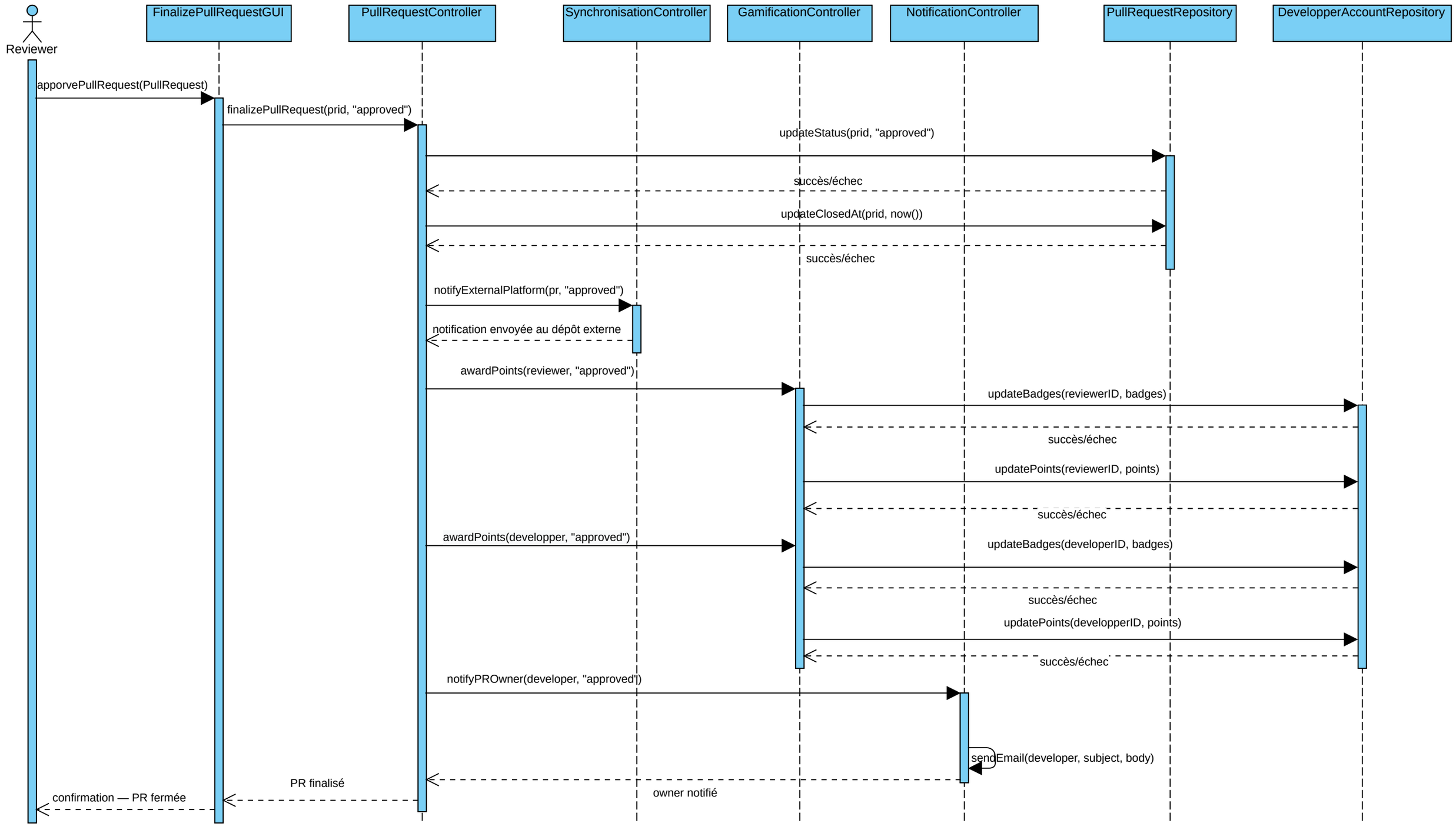
- Finaliser une PR: Ce cas est le plus riche en termes d'interactions : il déclenche une cascade d'opérations impliquant la synchronisation avec le dépôt externe, la notification du propriétaire de la PR par email, et l'attribution de points via le système de gamification.

Ces quatre scénarios couvrent la majorité des acteurs du système (Développeur, Reviewer, Dépôt externe) et traversent les quatre couches de l'architecture (Interface → Contrôleur → Repository → Entité), offrant ainsi une vision complète du comportement du système.

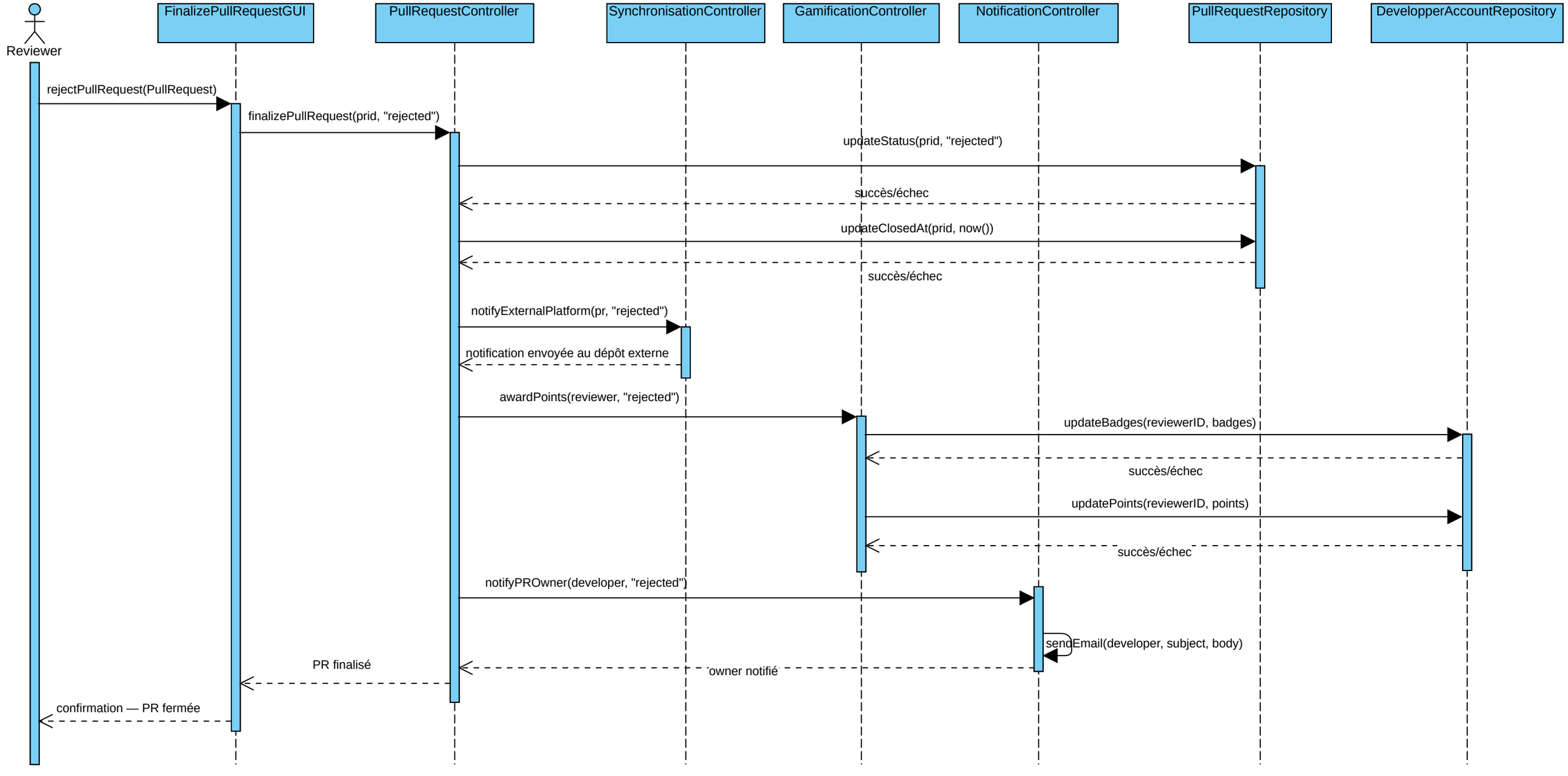

Login
User
LoginGUI
AuthenticationController
AccountRepository
submitform
login(email, password)
exists(email, password)
accountStatus
alt
[accountStatus=True]
verifyCredentials(email, password)
alt
[Verification=True]
findByEmail()
account
Login success
alt
[adminLogin]
redirectAdminDashboard()
[developperLogin]
redirectDashboard()
[Verification=False]
Login Failure
[accountStatus=False]
Login Failure
User
LoginGUI
AuthenticationController
AccountRepository


signUp
User
signUpGUI
AuthenticationController
DevelopperAccountRepository
ExternalPlatform
externalAuthRedirect()
linkExternalAccount(platformName, credentials)
authenticate(creds)
link
exists(link)
exists
alt
[exists=True]
signUp Failure
[exists=False]
addExternalAccountLink(uid: int, link: String)
linkAdded
accountLinked
submitform
signUp(accountData)
exists(email, password)
alt
[exists=True]
signUp Failure
[exists=False]
create(accountData)
DevelopperAccount
signUp success
redirectDashboard()
User
signUpGUI
AuthenticationController
DevelopperAccountRepository
ExternalPlatform

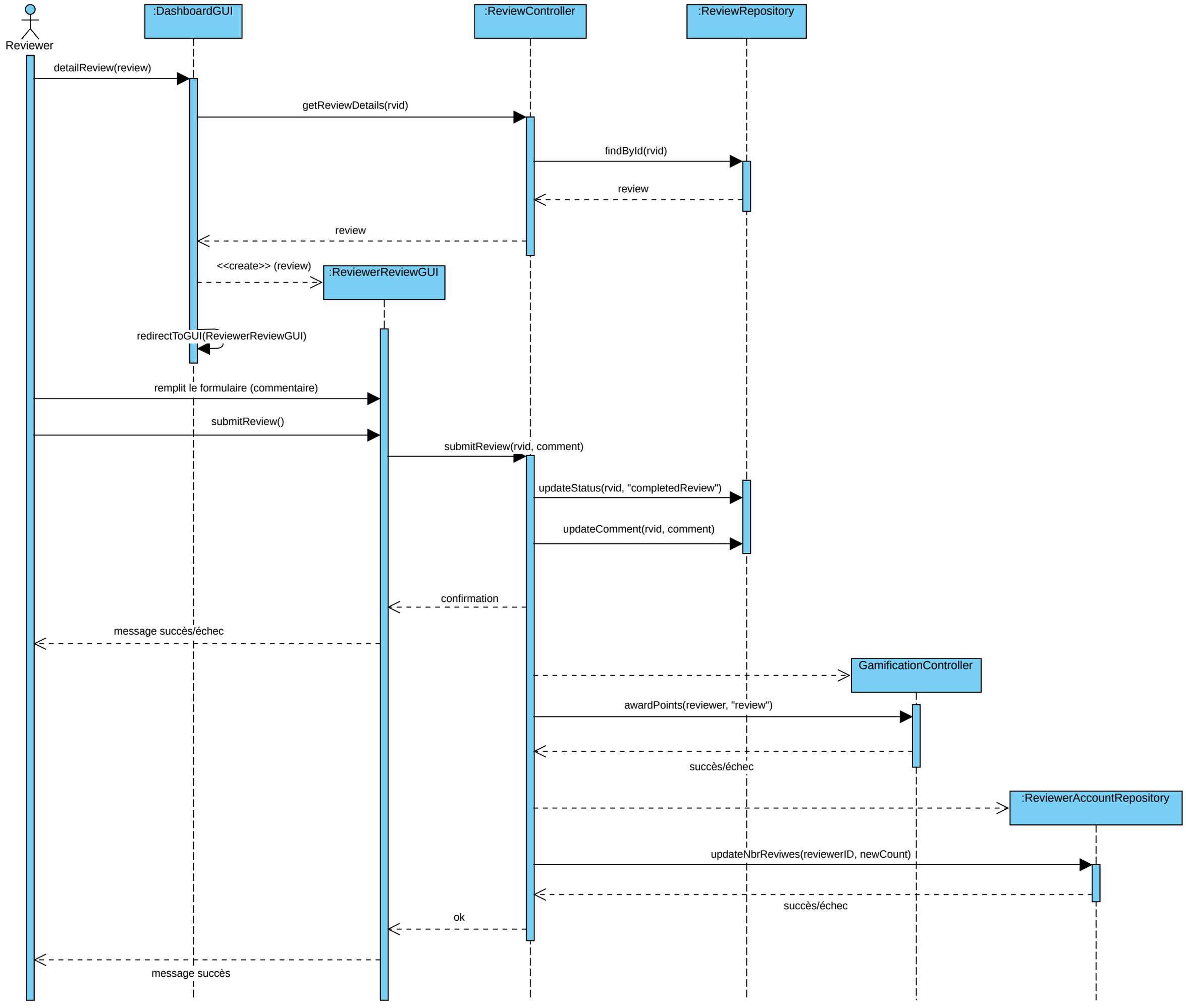
Approuver Pull Request



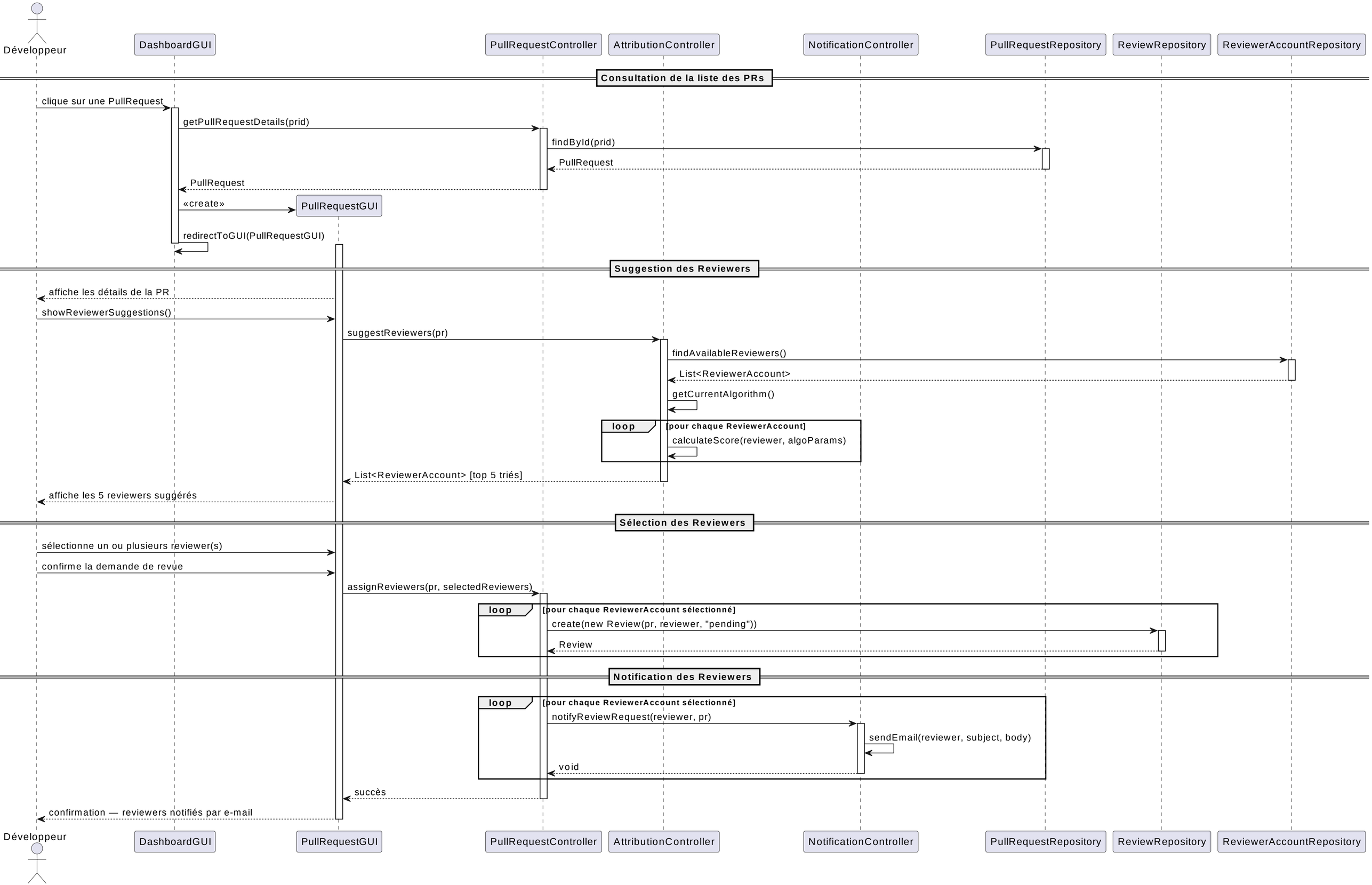
Rejeter Pull Request



Effectuer une revue d'une PR



Revendiquer une revue d'un Pull Request



V- Modélisation des processus métier

Après avoir modélisé les interactions entre objets via les diagrammes de séquence, on souhaite se centrer sur les processus : plutôt que de montrer qui communique avec qui, on s'intéresse au flux logique des opérations, aux décisions prises à chaque étape et aux parallélismes possibles.

Les diagrammes d'activité UML permettent de représenter ces workflows sous forme de graphes d'activités, incluant des nœuds de décision, des boucles et des barres de synchronisation (fork/join). Ils sont particulièrement adaptés pour :

- détailler la logique interne d'un algorithme complexe,
- donner une vue transversale d'un processus impliquant plusieurs acteurs,
- modéliser les scénarios alternatifs avec leurs points d'échec.

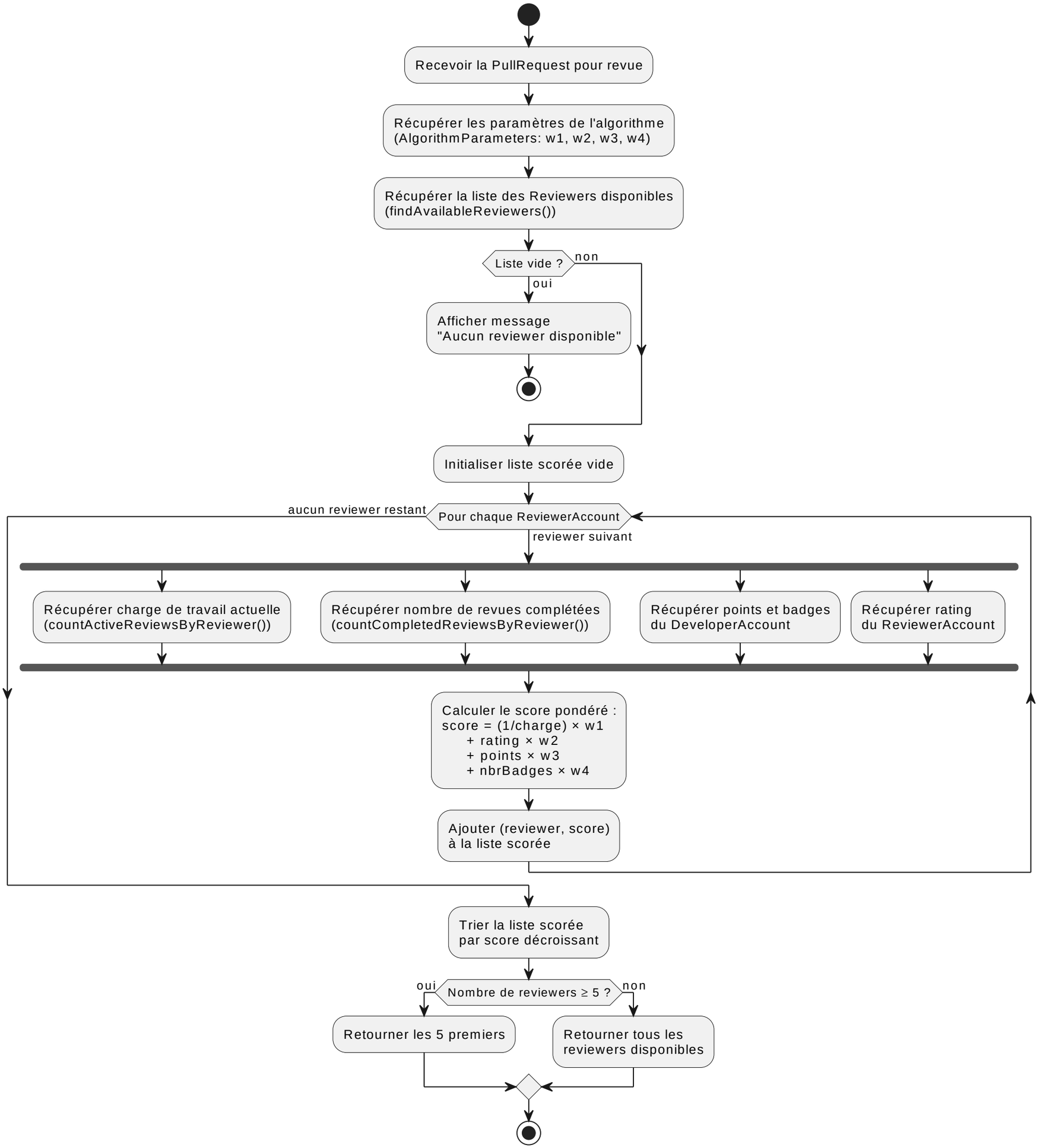
Nous avons sélectionné deux processus complémentaires :

1) Algorithme d'attribution des Reviewers

Ce diagramme détaille la logique interne de la méthode suggestReviewers() du contrôleur AttributionController, qui constitue le cœur fonctionnel de la plateforme.
Dans le diagramme de séquence du cas Revendiquer une revue d'une PR, cet algorithme apparaissait comme un simple appel de méthode dans une boucle. Le diagramme d'activité permet d'en expliciter le fonctionnement étape par étape : récupération parallèle des données de chaque reviewer (charge de travail, historique, points, rating),

calcul du score pondéré à l'aide des paramètres configurables de AlgorithmParameters, tri et sélection des 5 meilleurs candidats.

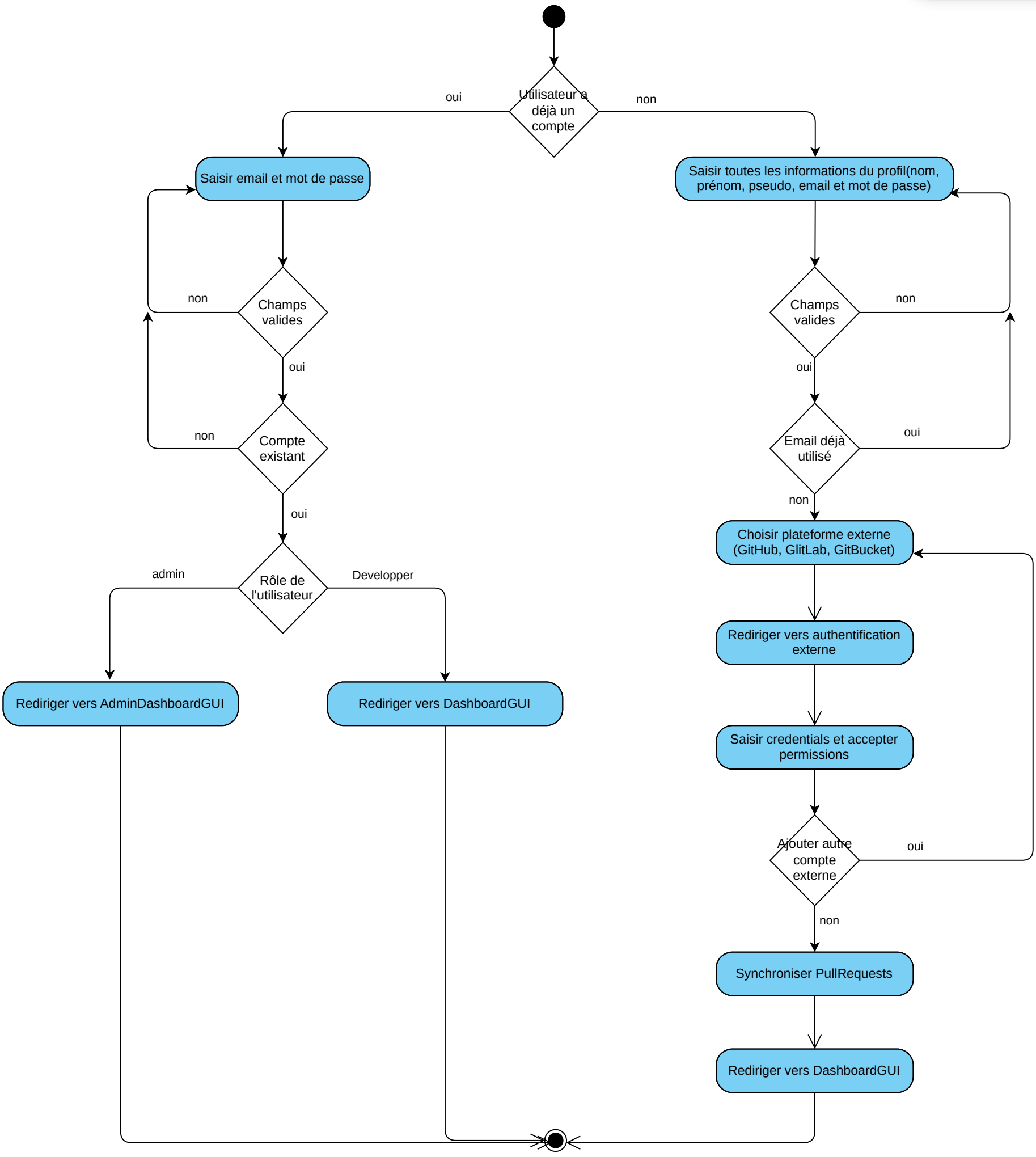
Diagramme d'activité — Algorithme d'attribution des Reviewers



2) Processus d'authentification (Login / SignUp)

Ce diagramme modélise les deux parcours possibles lors de l'entrée dans le système : la connexion avec un compte existant (Login) et la création d'un nouveau compte (SignUp). Il met en évidence la boucle d'ajout de comptes externes (GitHub, GitLab, Bitbucket) via OAuth, ainsi que les trois points d'échec possibles (email invalide, email déjà existant, échec de l'authentification OAuth) et les mécanismes de reprise associés.

S'authentifier (LogIn / SignUp)



VI- Modélisation des états

Les diagrammes précédents (séquence et activité) montrent le comportement du système du point de vue des interactions et des processus. Cette section s'intéresse à un angle complémentaire : le comportement d'un objet individuel au cours de sa vie dans le système.

Les diagrammes états-transitions UML modélisent les différents états qu'un objet peut traverser, les événements qui déclenchent les transitions entre ces états, et les actions exécutées lors de chaque transition. Ils sont particulièrement pertinents pour les objets dont le statut évolue de manière significative et conditionne le comportement du système.

Dans notre plateforme, deux entités possèdent un cycle de vie riche justifiant cette modélisation :

1) Diagramme états-transitions de PullRequest

Une Pull Request traverse les états suivants au cours de son cycle de vie :

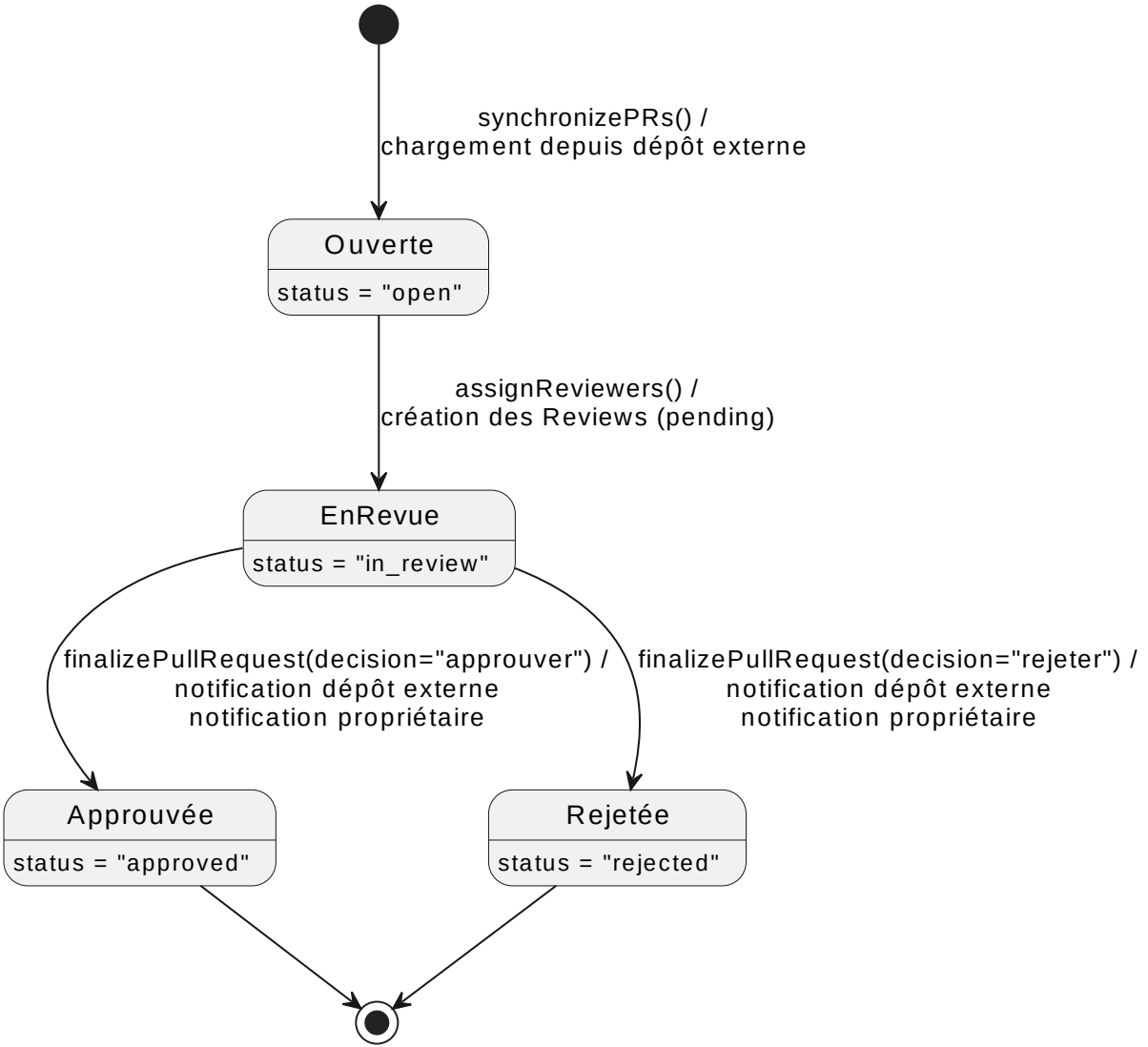
- Open : état initial, la PR vient d'être synchronisée depuis le dépôt externe. Elle est visible dans le tableau de bord et peut être revendiquée par un développeur.
- In_review : une ou plusieurs revues ont été assignées. La PR est en cours d'examen par les reviewers désignés.

- Approved : état final positif, le reviewer a approuvé la PR. Le dépôt externe et le propriétaire sont notifiés, et des points sont attribués au reviewer et au développeur.
- Rejected : état final négatif, le reviewer a rejeté la PR. Le dépôt externe et le propriétaire sont notifiés, et des points sont attribués au reviewer uniquement.

Les transitions sont déclenchées par les méthodes des contrôleurs :

- synchronizePRs() → crée la PR à l'état Open
- assignReviewers() → passage de Open à In_review
- finalizePullRequest(decision="approuver") → passage à Approved
- finalizePullRequest(decision="rejeter") → passage à Rejected

Diagramme État-Transition : PullRequest



2) Diagramme états-transitions de Review

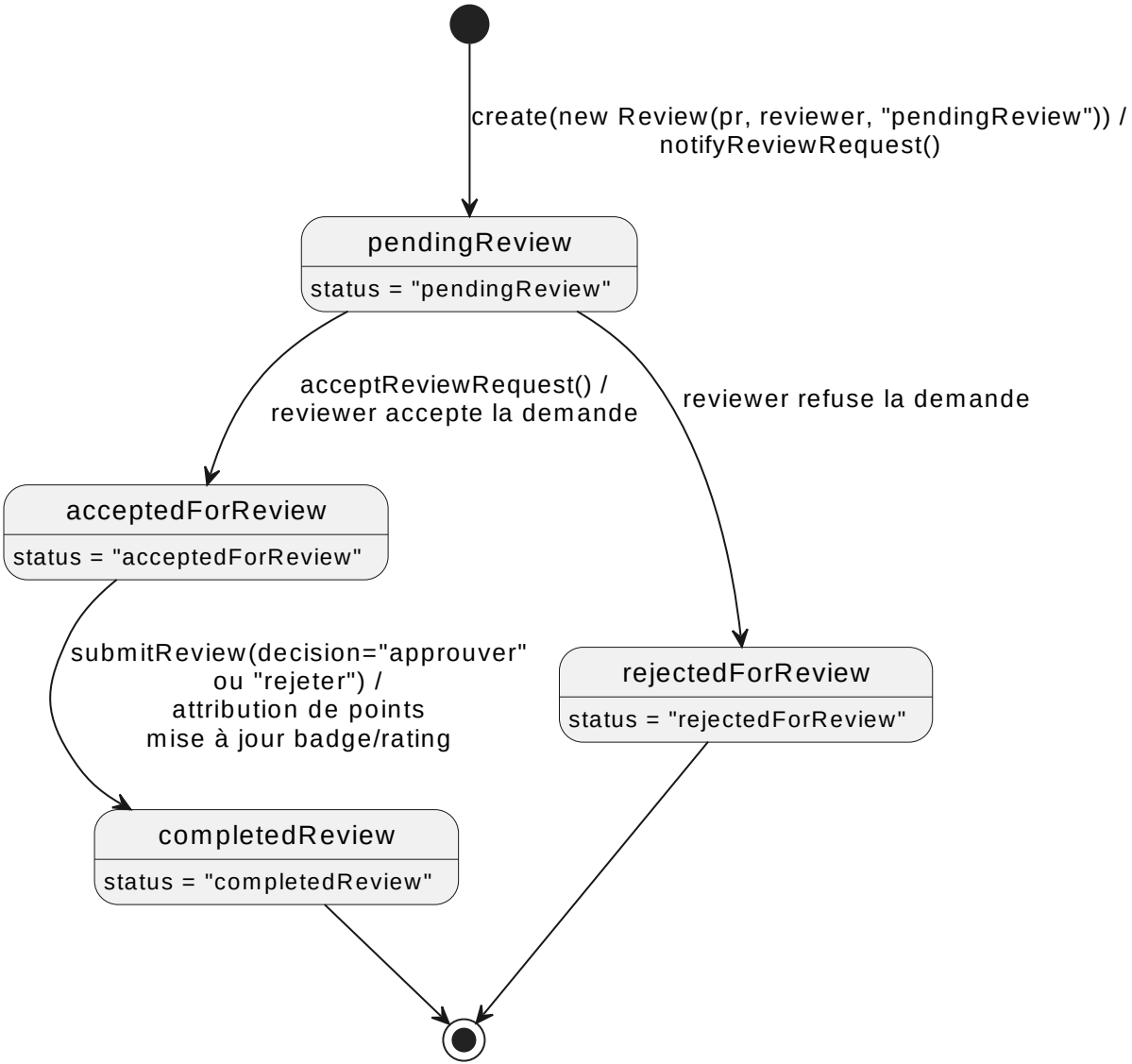
Une Review suit un cycle de vie en quatre états, définis par l'énumération ReviewStatus :

- pendingReview : état initial, la revue vient d'être créée suite à l'assignation d'un reviewer. Le reviewer est notifié par email et la revue apparaît dans sa liste de revues demandées.
- acceptedForReview : le reviewer a accepté de prendre en charge la revue. Il peut désormais consulter le code et rédiger son commentaire.
- rejectedForReview : le reviewer a décliné la demande de revue. Le développeur est notifié et peut choisir un autre reviewer.
- completedReview : état final, le reviewer a soumis son commentaire et sa décision. Des points lui sont attribués via le système de gamification.

Les transitions sont déclenchées par :

- create(new Review(...)) + notifyReviewRequest() → création à l'état pendingReview
- acceptReviewRequest() → passage à acceptedForReview
- refus du reviewer → passage à rejectedForReview
- submitReview() → passage à completedReview avec attribution de points et mise à jour des badges

Diagramme État-Transition : Review



VII- Architecture technique

Cette dernière section décrit l'architecture logicielle et physique du système, nous décrivons ici avec quoi et où le système s'exécute.

1) Diagrammes de composants

Le diagramme de composants représente l'organisation logicielle du système en modules cohérents et leurs dépendances. Il traduit le diagramme de packages en une vue plus concrète, orientée déploiement.
Notre système s'organise en quatre couches internes et trois systèmes externes :

Couches internes :

- Interface (GUI) : regroupe les 11 classes d'interface (LoginGUI, SignUpGUI, DashboardGUI, etc.) qui constituent le point de contact avec l'utilisateur.
- Controller : contient les 9 contrôleurs qui orchestrent la logique applicative. Ce composant dépend à la fois de la couche Interface (qui lui délègue les actions utilisateur) et de la couche Repository (à laquelle il délègue la persistance).
- Repository : regroupe les 7 repositories qui assurent l'accès aux données. Chaque repository est spécialisé pour une entité métier.
- Entity : contient les 6 classes métier (Account, DeveloperAccount, ReviewerAccount, AdminAccount, PullRequest, Review, AlgorithmParameters) qui représentent le domaine.

Systèmes externes :

- Dépôts externes (GitHub, GitLab, Bitbucket) : sources des Pull Requests, communiquant avec le système via le SynchronisationController.
- Service Email : utilisé par le NotificationController pour envoyer les notifications aux développeurs et reviewers.
- Base de données : stockage persistant des entités, accessible uniquement via les repositories.

Les dépendances respectent le principe de séparation en couches : l'interface ne communique qu'avec les contrôleurs, les contrôleurs ne communiquent qu'avec les repositories, et les repositories ne communiquent qu'avec les entités et la base de données.

Diagramme de composants — Interface (GUI) → Controller

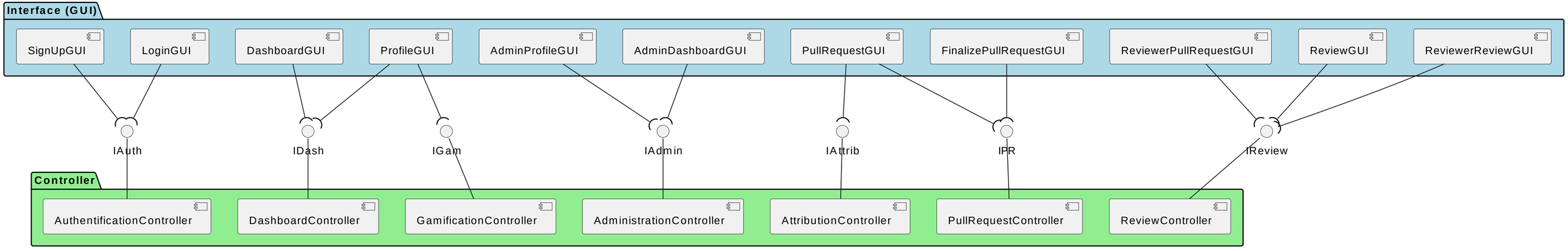


Diagramme de composants — Controller → Repository

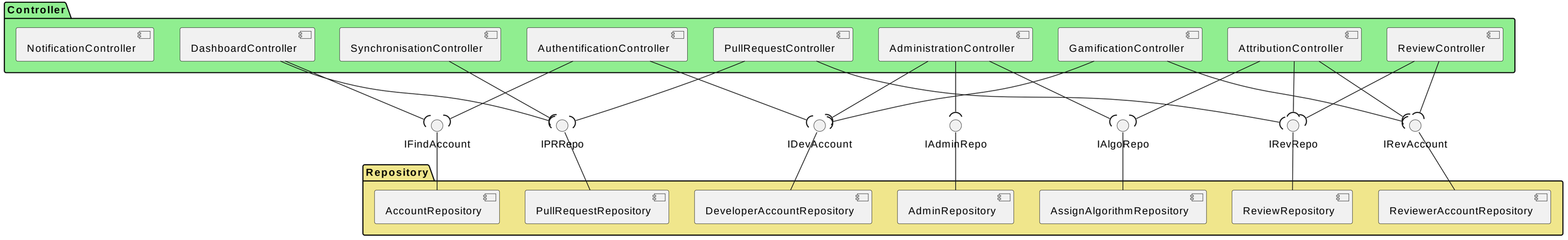


Diagramme de composants — Intégrations externes

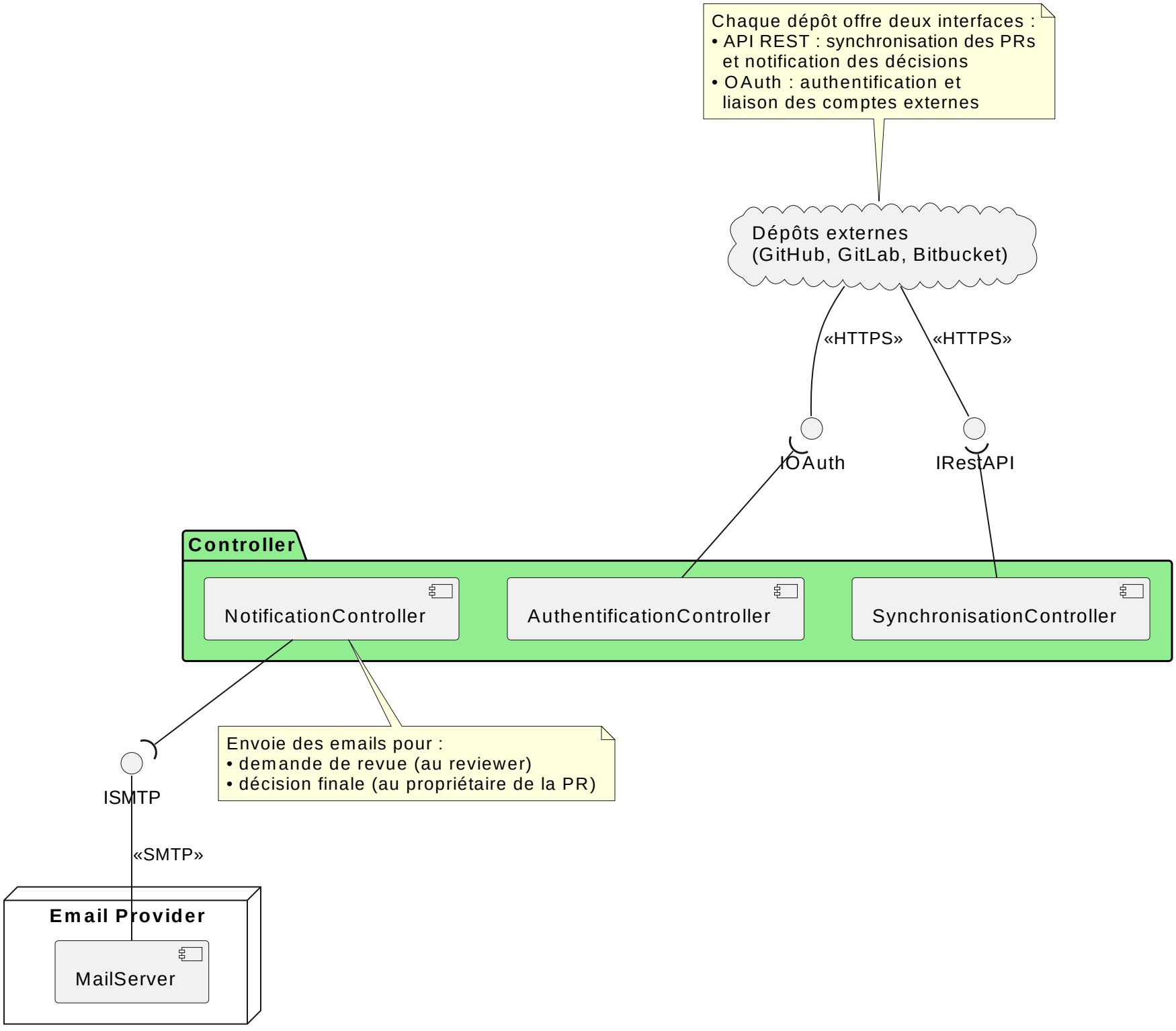


Diagramme de composants — Repository → Base de données

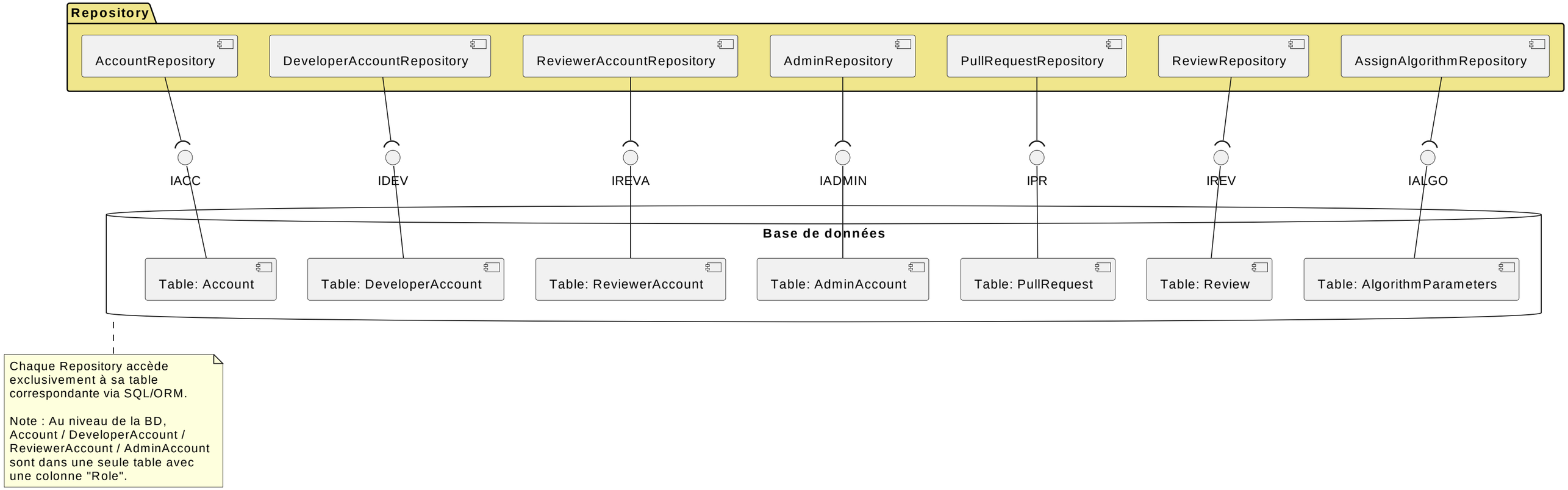
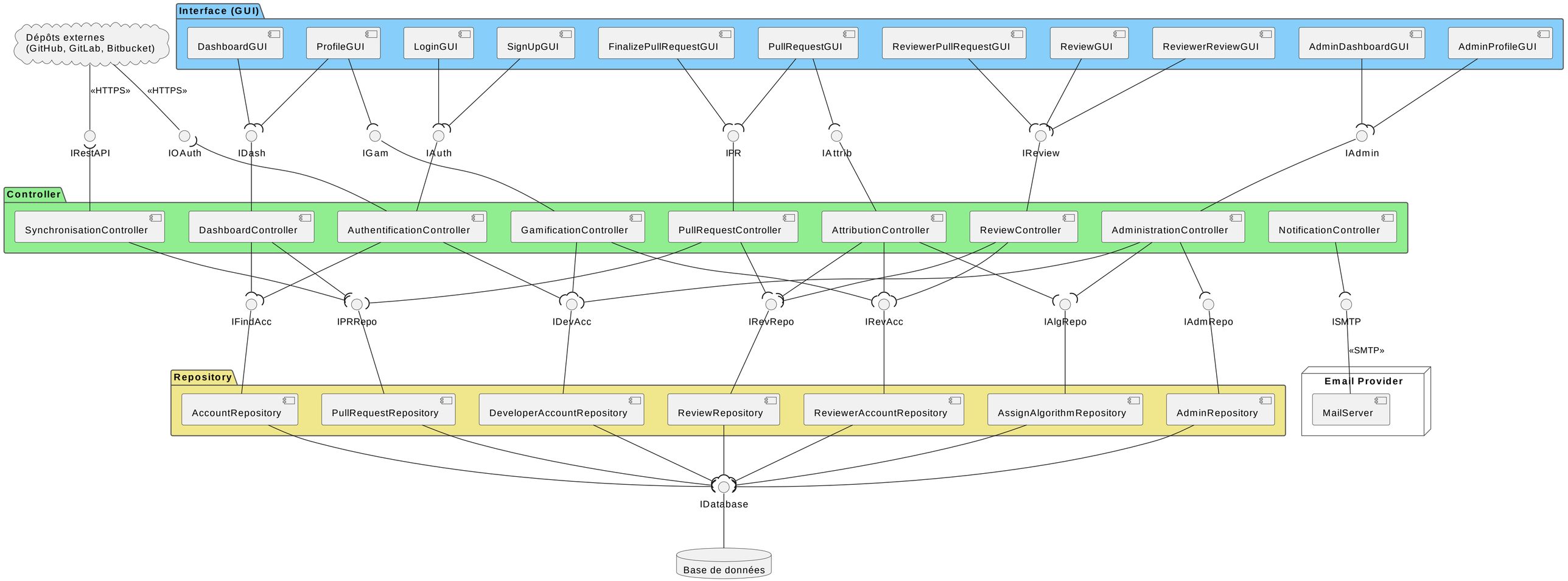


Diagramme de composants — Vue générale



2) Diagramme de déploiement

Le diagramme de déploiement décrit la topologie physique du système : les nœuds matériels (machines, serveurs), les composants logiciels déployés sur chaque nœud, et les protocoles de communication entre eux.

Notre architecture se décompose en cinq nœuds :

- Client : la machine de l'utilisateur (navigateur web ou application desktop) hébergeant la couche GUI. Il communique avec le serveur d'application via HTTPS.
- Serveur d'application : le serveur central hébergeant les couches Controller, Repository et Entity. Il constitue le cœur du système et assure la coordination entre toutes les parties. Il communique avec :
    - la base de données (protocole interne),
    - les dépôts externes via HTTPS (APIs REST de GitHub, GitLab, Bitbucket),
    - le fournisseur email via SMTP.
- Base de données : le serveur de base de données hébergeant le stockage persistant des entités. Il est accessible uniquement depuis le serveur d'application, jamais directement depuis le client.
- Dépôts externes : les serveurs tiers (GitHub, GitLab, Bitbucket) qui hébergent les Pull Requests sources. La communication est bidirectionnelle : le système récupère les PRs (synchronisation) et notifie les décisions (finalisation).

- Email Provider : le serveur de messagerie (MailServer) utilisé pour l'envoi des notifications. La communication est unidirectionnelle : le système envoie des emails mais ne reçoit pas de réponses via ce canal.

Cette architecture respecte le modèle client-serveur classique (MVC) avec une séparation nette entre la présentation (client), la logique métier (serveur d'application) et les données (base de données).

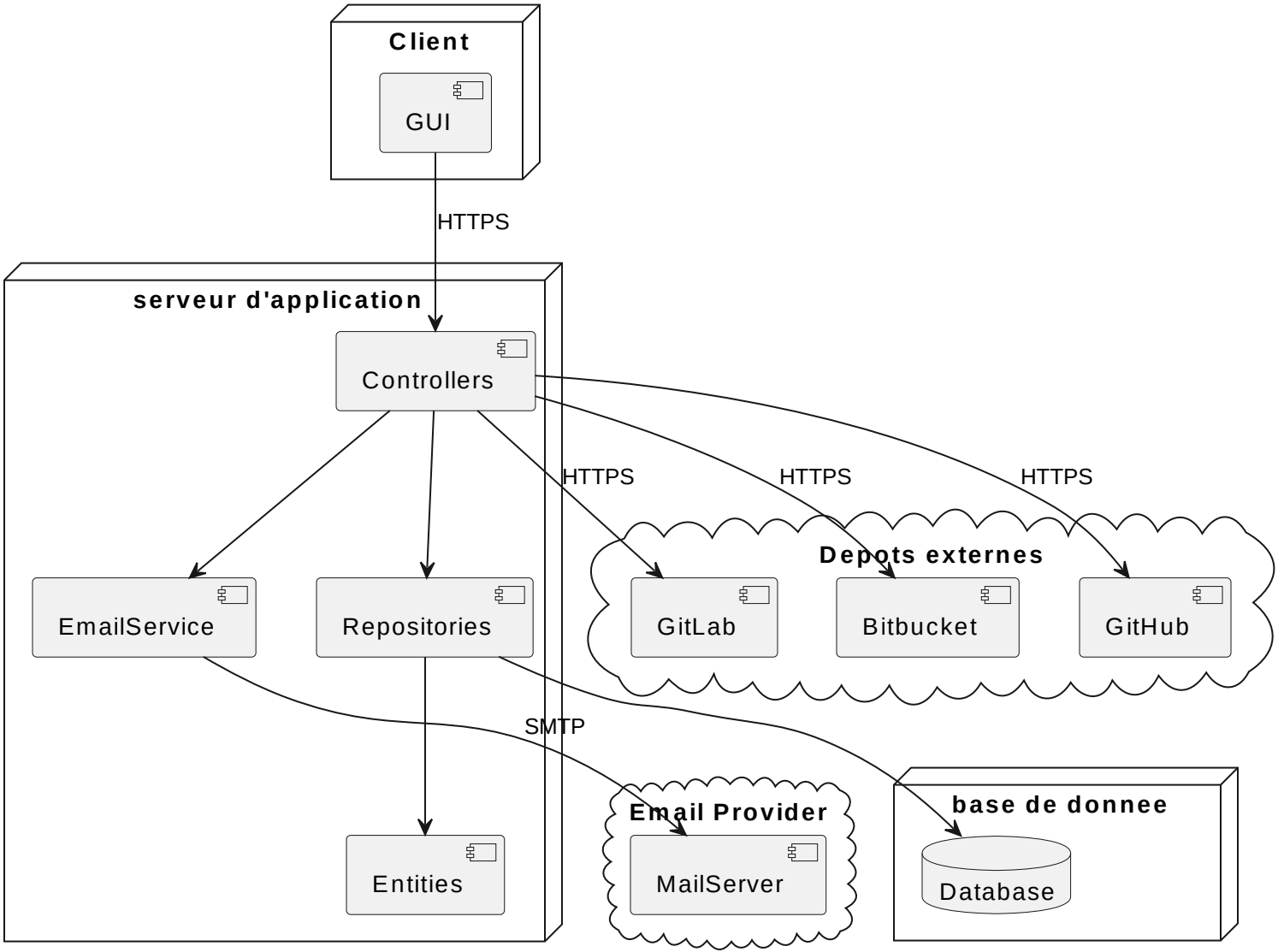



Diagramme de deploiment